

Trading Platform — Full System Architecture

How Every Piece Actually Works, Explained From Scratch

August 2026

1. The Big Picture
 - 1.1 The Five Programs
 - 1.2 How Data Flows, Start to Finish
2. Why Split Into So Many Pieces?
3. The C++ Risk Engine — The Fast Brain
 - 3.1 PriceStore — the original (now mostly retired) price cache
 - 3.2 OrderBook — the real, live price data
 - 3.3 UserStateStore — everyone's cash and positions, in RAM
4. Locks, Threads, and Your Multi-User Question
 - 4.1 What a "lock" (mutex) actually is
 - 4.2 Why UserStateStore uses two locks, not one
 - 4.3 Where the actual threads come from
 - 4.4 The part that *isn't* built yet
5. The gRPC Contract — How Python and C++ Actually Talk
6. Why Orders Don't Touch the Database Directly (WAL + Kafka)
 - 6.1 The problem
 - 6.2 The fix: write to RAM + a simple log file, catch up on the database later
 - 6.3 Step by step, following one single trade
 - 6.4 The honest cost of this design
7. Live Market Data — `orderbook_feed_producer.py`
8. The Strategy Layer (Built This Weekend)
 - 8.1 The interface — `strategies/base.py`
 - 8.2 The one strategy that exists — `ThresholdStrategy`
 - 8.3 The runner — the current single-threaded loop
9. The Dashboard — `streamlit_app.py`
 - 9.1 Login — two independent paths
 - 9.2 Auto-refresh without a full page reload
10. Direct Answer: Can This System Handle Multiple Strategies From Different Users?
11. Glossary

1. The Big Picture

Before touching any code, here's what this system is in one paragraph:

It's a paper-trading platform (fake money, real market data) where a piece of code called a **strategy** watches live Bitcoin prices from Binance and decides when to buy and sell. Every decision it makes gets checked by a risk system before it's allowed to happen, gets recorded permanently, and shows up on a live dashboard. The whole thing is split across five different programs, written in two different languages, talking to each other over a network — even though right now they might all be running on the same laptop.

That last sentence is the first thing worth understanding: **why is this five separate programs instead of one?** Every design decision below traces back to that question.

1.1 The Five Programs

#	Program	Language	Job
1	risk_engine.exe	C++	Holds live prices and everyone's cash/positions in RAM. Approves or rejects every order in microseconds.
2	Flask API (app.py)	Python	The web server. Handles login, registration, and (in the past) manual order placement over HTTP.
3	orderbook_feed_producer.py	Python	Connects to Binance over WebSocket, forwards live prices into the C++ engine.
4	strategy_runner.py	Python	The "trader." Polls prices, runs the strategy, submits orders.
5	streamlit_app.py	Python	The dashboard you look at in a browser.

Plus two infrastructure pieces that aren't "our code" but everything depends on:

- **PostgreSQL** — the real database. Permanent storage: who has what account, what orders happened, what the ledger says.
- **Kafka** (via Redpanda/apache/kafka in Docker) — a message queue. A place to drop off a piece of information ("user 16 just bought 0.01 BTC") so another program can pick it up later, without the two programs having to talk to each other directly or wait on each other.

1.2 How Data Flows, Start to Finish

Binance (real exchange)

| live order book, WebSocket

▼

orderbook_feed_producer.py — gRPC —► risk_engine.exe (C++)

| ▲

holds prices + | gRPC calls:

everyone's cash/ | "can I buy this?"

positions in RAM | "here's what happened"

| |

▼ |

strategy_runner.py

(the "trader")

|

| writes one line per fill

▼

wal.log (a plain text file)

|

| tailed by

▼

wal_producer.py —► Kafka topic "order-fills"

|

| consumed by

▼

fill_consumer.py

|

| writes real rows

▼

PostgreSQL

(accounts, positions, orders)

|

▼

streamlit_app.py

(the dashboard you watch)

Every arrow in that diagram is a real, separate hop — a network call, a file write, or a database write. None of it happens “in one step.” That’s deliberate, and Section 6 explains exactly why.

2. Why Split Into So Many Pieces?

Three different reasons, three different splits:

Reason 1 — Speed. Checking “does this person have enough money to buy this?” needs to happen in *microseconds*, because in a real exchange thousands of these checks happen every second. Python is not built for that kind of raw speed. C++ is. So the one piece of the system that has to be blazingly fast (`risk_engine.exe`) is written in C++, and everything else — which cares more about being easy to write and change than about raw speed — is Python.

Reason 2 — Not blocking on slow things. Writing to a database takes milliseconds, not microseconds — a thousand times slower than the RAM-only check above. If every single order had to wait for a database write to finish before responding, the system would be a thousand times slower than it needs to be. So the fast path (checking and approving an order) never touches the database directly. It writes to RAM and to a simple log file, and a *separate* slower pipeline (Kafka → Postgres) catches up in the background. Section 6 walks through this in detail — it’s one of the more subtle ideas in the whole system.

Reason 3 — Decoupling. If `strategy_runner.py` crashes, you don’t want `streamlit_app.py` to crash too. If Binance’s connection drops, you don’t want the risk engine to stop responding to orders that are already in flight. Keeping these as separate processes means one crashing doesn’t take the others down with it.

3. The C++ Risk Engine — The Fast Brain

This is `risk_engine.exe`. It does four things, and nothing else:

1. Remembers the current price of every symbol
2. Remembers every trader’s cash and positions
3. Decides “yes you can place this order” or “no you can’t” in microseconds
4. Writes down what happened so it isn’t lost

It's a single program that stays running the whole time, holding everything **in RAM** (not on disk) so that reading/checking is as fast as physically possible. Let's go piece by piece.

3.1 PriceStore — the original (now mostly retired) price cache

Originally, this system simulated stock prices with a random-walk formula (geometric Brownian motion — a common way to fake realistic-looking price movement). PriceStore is that original mechanism:

```
// price_store.hpp

class PriceStore {
public:
    void set_price(const std::string& symbol, double price, const std::string& updated_at);
    void add_symbol(const std::string& symbol, double price, double volatility, double drift, const std::string& updated_at);

    std::optional<PriceData> get_symbol_data(const std::string& symbol) const;

    std::unordered_map<std::string, double> get_all() const;

private:
    mutable std::shared_mutex shared_mutex;

    std::unordered_map<std::string, PriceData> prices;
};
```

It's still in the code and still gets checked as a fallback, but the simulated price thread (`run_gbm`) is currently switched off in `main.cpp` — real Binance data replaced it. It's kept around as a safety net: if a symbol has no live order-book data yet, the engine falls back to whatever's in here instead of failing outright.

3.2 OrderBook — the real, live price data

This is what actually drives prices today. Every ~100ms, `orderbook_feed_producer.py` pushes a fresh snapshot of real Binance bid/ask levels into this store.

```
// order_book.hpp

struct PriceLevel { double price; double quantity; };

struct OrderBookEntry {
    std::vector<PriceLevel> bids;
    std::vector<PriceLevel> asks;
    std::string updated_at;
    std::unique_ptr<std::mutex> lock;
};

class OrderBook {
public:
    void update(const std::string& symbol, const std::vector<PriceLevel>& bids,
               const std::vector<PriceLevel>& asks, const std::string& updated_at);

    std::optional<double> best_bid(const std::string& symbol) const;
    std::optional<double> best_ask(const std::string& symbol) const;
    ...
};
```

“Bid” and “ask”, quickly, if that’s new: the *bid* is the highest price someone is currently willing to *buy* at. The *ask* is the lowest price someone is currently willing to *sell* at. There’s always a small gap between them (the *spread*). When you place a market BUY order, you pay the ask; when you SELL, you receive the bid.

3.3 UserStateStore — everyone’s cash and positions, in RAM

This is the one that matters most for the multi-user question later in this document, so read this one closely.

```
// user_state_store.hpp

struct UserState {
    double cash;

    std::unique_ptr<std::mutex> lock;    // <-- one Lock PER USER

    std::unordered_map<std::string, PositionState> positions;
};

class UserStateStore {
    ...

private:
    std::unordered_map<int, UserState> user_positions;

    std::shared_mutex user_positions_mutex;    // <-- one Lock for the WHOLE MAP
};
```

Notice there are **two different locks here**, at two different levels. This is the key design idea of the whole engine, so let's slow down on it.

4. Locks, Threads, and Your Multi-User Question

You asked whether this system can actually run multiple people's strategies at once, on different threads, without them stepping on each other. Short answer: **the C++ engine is already built for exactly that. The Python side that would drive it isn't, yet.** Here's precisely what that means.

4.1 What a “lock” (mutex) actually is

A mutex (mutual exclusion) is a way of saying “only one thread may touch this piece of data at a time.” If two threads try to change the same number in memory at the exact same instant, you can get corrupted, nonsensical results (this is called a *race condition*). A mutex prevents that by making the second thread wait its turn.

The catch: if you protect *everything* with one single lock, you’ve accidentally made your “concurrent” program behave like it’s single-threaded again — every thread just queues up single-file, waiting for the one lock. That defeats the whole point of having multiple threads.

4.2 Why UserStateStore uses two locks, not one

Look again at the class:

```
std::unordered_map<int, UserState> user_positions;  
std::shared_mutex user_positions_mutex;
```

- user_positions_mutex protects the **map itself** — the structure that says “user 16’s data lives here in memory.” You only need this lock very briefly, just to *find* the right user.
- Each individual UserState has **its own separate lock** (state.lock, a std::unique_ptr<std::mutex>), protecting *that one user’s* cash and positions specifically.

Here’s the actual order-checking function:


```

// user_state_store.cpp

ReservationResult UserStateStore::check_and_reserve_position(
    int user_id, const std::string& side, const std::string& symbol,
    double quantity, double price)
{
    std::shared_lock lock{user_positions_mutex};    // 1. briefly Lock the MAP

    if (user_positions.find(user_id) == user_positions.end()) {
        return {false, 0, 0, 0};
    }

    UserState& user = user_positions[user_id];

    std::unique_lock lock2{*user.lock};            // 2. Lock THIS USER ONLY
    lock.unlock();                                // 3. release the map lock immediately

    if (side == "BUY") {
        if (user.cash >= quantity * price) {
            user.cash -= quantity * price;
            // ... update position ...
            return {true, user.cash, ...};
        }
        return {false, ...};
    }
    // ... SELL is symmetric ...
}

```

Walk through what actually happens when **two different users** place orders at the same moment:

1. Both threads briefly grab the map lock, one after the other, just long enough to find their own UserState in the map (this step is nearly instant).
2. Both threads then grab their **own** user's individual lock and immediately release the shared map lock (lock.unlock() on the very next line).
3. From this point on, **user A's thread and user B's thread are running completely independently**. Neither is waiting on the other. User A's cash/position math and user B's cash/position math happen genuinely in parallel.

The *only* time two threads actually wait on each other is if they're both trying to trade **the exact same user's account** at the exact same instant — which is correct and necessary (you don't want two orders for the same person corrupting each other's math), and which will be rare in practice.

OrderBook uses the identical pattern for a different reason — one lock per **symbol** instead of per **user** — so that a price update for BTC doesn't block a price read for ETH.

4.3 Where the actual threads come from

None of this matters unless something is actually *calling* these functions from multiple threads at once. That something is the gRPC server itself. In `main.cpp`:

```
grpc::ServerBuilder builder;
builder.AddListeningPort("0.0.0.0:50051", grpc::InsecureServerCredentials());
builder.RegisterService(&service);
std::unique_ptr<grpc::Server> server = builder.BuildAndStart();
server->Wait();
```

By default, a C++ gRPC server like this handles incoming requests using an internal pool of worker threads — when two `CheckOrder` requests arrive from two different clients at close to the same time, gRPC hands them to two different threads automatically, without you writing any thread-management code yourself. Those two threads then call into `UserStateStore`, and the per-user locking described above is precisely what makes that safe.

So: yes, the engine can already process two different users' orders genuinely concurrently, on separate threads, safely. This part of your instinct was correct, and it was clearly a deliberate design choice on your part — the per-user mutex only makes sense if you were already thinking about concurrent access from the start.

4.4 The part that *isn't* built yet

Here's the honest gap. `strategy_runner.py` — the Python program that actually decides when to buy/sell and calls the engine — is currently a single `while True:` loop in a single thread:

```
# strategy_runner.py (current shape)

while True:

    price = app.risk_engine.get_price(symbol)["price"]

    for intent in strategy.on_price(symbol, price):

        app.order_service.place_order(user_id=user.id, ...)

    time.sleep(POLL_INTERVAL_SECONDS)
```

There's only ever one `user_id` in this loop right now (`strategy_bot`). If you pointed this same loop at ten users, it would check them **one after another**, not at the same time — even though the C++ engine underneath is perfectly capable of handling all ten concurrently. The concurrency capability exists in the part of the system built to need it (the risk engine, which has to survive real concurrent load from a real exchange). It doesn't yet exist in the part that was, until this weekend, only ever driving one hardcoded account.

Closing this gap — making `strategy_runner.py` actually take advantage of the concurrency that's already sitting there in the C++ layer — is exactly the “multi-account” work discussed in the next planning document.

5. The gRPC Contract — How Python and C++ Actually Talk

Python and C++ can't call each other's functions directly — they're different languages, different compiled binaries, potentially different machines. **gRPC** solves this: you write a `.proto` file describing exactly what requests and responses look like, and gRPC generates matching code in both languages that knows how to serialize a request, send it over the network, and deserialize the response.

```
// trading.proto (excerpt)

message CheckOrderRequest {
    int32 user_id    = 1;
    string order_id   = 2;
    string symbol     = 3;
    string side       = 4;
    string order_type = 5;
    double quantity   = 6;
    double limit_price = 7;
}

message CheckOrderResponse {
    bool approved = 1;
    string reason  = 2;
    double fill_price = 4;
    double new_cash_balance = 5;
    double new_average_price = 6;
    double new_quantity = 7;
}

service TradingService {
    rpc CheckOrder    (CheckOrderRequest) returns (CheckOrderResponse);
    rpc UpdateState   (UpdateStateRequest) returns (UpdateStateResponse);
    rpc LoadUser      (LoadUserRequest)   returns (LoadUserResponse);
    rpc GetPrice       (GetPriceRequest)    returns (GetPriceResponse);
    rpc GetAllPrices   (GetAllPricesRequest) returns (GetAllPricesResponse);
    rpc HasUser        (HasUserRequest)     returns (HasUserResponse);
    rpc UpdateOrderBook (OrderBookUpdateRequest) returns (OrderBookUpdateResponse);
}
```

Each `rpc` line is one function that Python can call as if it were local, and C++ implements on the other end. The full list, and what each one is *for*:

RPC	Called by	Purpose
GetPrice / GetAllPrices	Python	“What’s the current price of X?”
CheckOrder	Python	“Can this user afford this order? What would the fill price be?” — doesn’t commit anything yet
UpdateState	Python	“This order was approved and executed — update your in-memory records and write it to the log.”
LoadUser	Python	“This user isn’t in your memory yet — here’s their cash/positions from the database, load them in.”
HasUser	Python	“Is this user already loaded in memory?” (avoids an unnecessary database read on every single order)
UpdateOrderBook	orderbook_feed_producer.py	“Here’s a fresh snapshot of real Binance bid/ask levels for this symbol.”

6. Why Orders Don’t Touch the Database Directly (WAL + Kafka)

This is the trickiest idea in the system, so it gets its own full walkthrough.

6.1 The problem

Writing to PostgreSQL takes a few milliseconds. That sounds fast, but compared to a RAM-only check (a fraction of a millisecond), it’s enormous — a thousand times slower. If placing an order meant “check risk in RAM, *then wait for a database write to finish*, then respond,” every single order would be bottlenecked by the slowest part of that chain, even though 99% of the work (the actual risk check) was already done almost instantly.

6.2 The fix: write to RAM + a simple log file, catch up on the database later

Instead, the moment an order is approved and filled:

1. The C++ engine updates its own in-memory numbers immediately (fast — this is the `UserStateStore` update from Section 4).
2. It writes **one line of text** describing what happened to a file called `wal.log` (WAL = **Write-Ahead Log** — a very old, well-proven database technique: write down what you're about to do, or just did, before/as you do it, so nothing is lost even if something crashes right after).
3. It responds to Python immediately. The order is done, as far as the trader is concerned.
4. **Completely separately, later, unhurried**, a chain of two small programs picks that line up and eventually writes it into PostgreSQL properly.

That fourth step is where Kafka comes in — not because it's fast, but because it lets step 4 happen *whenever*, without step 3 waiting around for it.

6.3 Step by step, following one single trade

Step A — the WAL entry gets written (`wal_writer.cpp`, called from inside the C++ engine):

```
WALWriter::WALWriter(const std::string& path) : file(path, std::ios::app) {}

void WALWriter::write_fill(int user_id, const std::string& order_id, ...)
{
    std::ostringstream ss;
    ss << "{\"type\":\"FILL\"";
    ss << ",\"user_id\":\"" << user_id
    ss << ",\"order_id\":\"" << order_id << "\"";
    // ... every other field ...
    ss << "}\n";

    std::lock_guard<std::mutex> guard(lock);
    file << ss.str();
    file.flush();
}
```

This appends one JSON object, on one line, to `wal.log`. That's it. No database involved at all at this point.

Step B — `wal_producer.py` notices the new line and forwards it to Kafka:

```

# pipeline/wal_producer.py

while True:
    if os.path.exists(WAL_PATH):
        with open(WAL_PATH, "r") as file:
            file.seek(position)          # pick up where we left off last time
            while True:
                line = file.readline()

                if not line:
                    break

                if line.strip():
                    data = json.loads(line)
                    producer.send(KAFKA_TOPIC, value=data)  # drop it into Kafka
                    producer.flush()
                    save_cursor(file.tell())                 # remember our position
                    position = file.tell()

            time.sleep(0.1)

```

This is a small standalone script whose entire job is: watch the file, and every time a new line shows up, forward it into a Kafka topic called `order-fills`. It remembers its position (`wal.cursor`) so a restart doesn't reprocess everything from scratch.

Step C — Kafka just holds it. This is the part that makes the whole system resilient. Kafka's job here is extremely simple: it's a durable, ordered queue. `wal_producer.py` drops messages in; something else picks them up whenever it's ready. If the "something else" (Step D) is temporarily down, the message just waits in Kafka — nothing is lost, nothing blocks.

Step D — `fill_consumer.py` reads from Kafka and finally writes to Postgres:

```

# pipeline/fill_consumer.py

def process_fill(fill_data):
    new_order = order.Order(
        id=fill_data["order_id"], user_id=fill_data["user_id"],
        symbol=fill_data["symbol"], side=fill_data["side"],
        quantity=fill_data["quantity"], filled_price=fill_data["fill_price"],
        status=OrderStatus.FILLED, order_type=fill_data["order_type"],
    )
    db.session.add(new_order)

    acc = account.Account.query.filter_by(user_id=fill_data["user_id"]).first()
    acc.cash_balance = fill_data["new_cash"]

    pos = position.Position.query.filter_by(user_id=fill_data["user_id"], symbol=fill_data["symbol"]).first()
    if pos:
        pos.quantity = fill_data["new_quantity"]
        pos.average_price = fill_data["new_avg_price"]
    else:
        db.session.add(position.Position(...))

    db.session.commit() # <-- the ONLY place in this whole chain that touches Postgres

def main():
    consumer = KafkaConsumer(KAFKA_TOPIC, ...)
    with app.app_context():
        for message in consumer:
            process_fill(message.value)

```

This is the one and only program that actually writes Order/Account/Position rows into PostgreSQL. Everything before it — the C++ engine, the WAL file, Kafka — was just getting the information here safely, without making the trader wait for any of it.

6.4 The honest cost of this design

There's a real trade-off here, and it's worth naming: **there's a short delay between “the trade happened” and “PostgreSQL knows about it.”** Usually well under a second, but it's not zero. This is why, earlier this session, the dashboard briefly looked “stuck” at the starting balance even though trades were firing — `wal_producer.py` had actually crashed (a malformed line it didn't know how to handle safely), so step B never happened for anything after that point, and nothing downstream ever caught up. Once that was fixed and it reprocessed the backlog, everything caught up correctly. This kind of design buys you speed and resilience at the cost of “everything is eventually correct” rather than “everything is instantly correct” — a very common, very deliberate trade-off in real trading systems, not a flaw unique to this one.

7. Live Market Data — `orderbook_feed_producer.py`

This program's only job is getting real prices from Binance into the C++ engine.

```
def discover_symbols():
    """Every actively-trading USDT pair from Binance."""
    with urllib.request.urlopen(BINANCE_EXCHANGE_INFO_URL, timeout=30) as response:
        payload = json.loads(response.read())
    return sorted(entry["symbol"] for entry in payload["symbols"]
                  if entry["quoteAsset"] == "USDT" and entry["status"] == "TRADING")

def build_stream_url(symbols):
    streams = "/".join(f"{symbol.lower()}@depth20@100ms" for symbol in symbols)
    return f"wss://stream.binance.com:9443/stream?streams={streams}"
```

It asks Binance “what's currently tradeable,” then opens a WebSocket subscription asking for the top 20 order-book levels for every one of those symbols, updated every 100 milliseconds. Because there are hundreds of symbols and a single connection can't reasonably carry all of them, it splits the list into batches (“shards”) of 100 and opens one WebSocket connection per batch, each running on its own thread:

```
def run_shard(shard_id, symbols, stub):  
    while True:  
        ws_app = websocket.WebSocketApp(stream_url, on_message=on_message, ...)  
        ws_app.run_forever()  
        print(f"[shard {shard_id}] Disconnected, reconnecting in 5s...")  
        time.sleep(5)
```

Every time a message arrives, it's translated into the `UpdateOrderBook` gRPC call from Section 5 and sent straight into the C++ engine's `OrderBook`. If a shard's connection drops, it automatically reconnects after 5 seconds rather than dying — so one bad connection doesn't take down price data for the other 400 symbols.

8. The Strategy Layer (Built This Weekend)

This is the newest part of the system — the part that turns “an exchange simulator” into “something a researcher can actually use.”

8.1 The interface — strategies/base.py

```
@dataclass
class OrderIntent:
    side: OrderSide
    symbol: str
    quantity: float
    order_type: OrderType = OrderType.MARKET
    limit_price: float | None = None

class Strategy(ABC):
    def __init__(self, symbols: list[str]) -> None:
        self.symbols = symbols

    @abstractmethod
    def on_price(self, symbol: str, price: float) -> list[OrderIntent]:
        raise NotImplementedError
```

The whole point of this shape: a Strategy only ever sees a price come in, and only ever hands back a list of “I want to trade this” requests. It knows nothing about gRPC, databases, or how orders actually get executed. That’s deliberate — it means the exact same strategy class could later be fed *historical* prices in a backtester instead of *live* prices here, with zero changes to the strategy itself.

8.2 The one strategy that exists — ThresholdStrategy

```
class ThresholdStrategy(Strategy):
    def on_price(self, symbol: str, price: float) -> list[OrderIntent]:
        reference = self.reference_price.get(symbol)

        if reference is None:
            self.reference_price[symbol] = price
            return []

        if not self.holding[symbol] and price <= reference * (1 - self.drop_pct):
            self.holding[symbol] = True
            self.reference_price[symbol] = price
            return [OrderIntent(side=OrderSide.BUY, symbol=symbol, quantity=self.trade_quantity)]

        if self.holding[symbol] and price >= reference * (1 + self.rise_pct):
            self.holding[symbol] = False
            self.reference_price[symbol] = price
            return [OrderIntent(side=OrderSide.SELL, symbol=symbol, quantity=self.trade_quantity)]

        return []
```

Buy the dip, sell the rip — not meant to be profitable, meant to prove the whole pipeline (live data → strategy → risk-checked order → fill → accounting → dashboard) actually works end to end.

8.3 The runner — the current single-threaded loop

```
def run() -> None:
    app = create_app()
    with app.app_context():
        user = get_or_create_strategy_user()
        strategy = ThresholdStrategy(symbols=SYMBOLS, drop_pct=DROP_PCT, ...)

        while True:
            for symbol in strategy.symbols:
                price = app.risk_engine.get_price(symbol)["price"]
                for intent in strategy.on_price(symbol, price):
                    order = app.order_service.place_order(user_id=user.id, ...)
            time.sleep(POLL_INTERVAL_SECONDS)
```

As covered in Section 4.4: right now this drives exactly one hardcoded account, sequentially, in one thread. This is the piece that needs to change to support multiple researchers with genuinely separate accounts.

9. The Dashboard — `streamlit_app.py`

Three jobs: gate access behind login, show the current state, refresh automatically.

9.1 Login — two independent paths

```
def require_login() -> None:
    if "username" in st.session_state or st.user.is_logged_in:
        return # already authenticated by one path or the other

    if st.button("Sign in with Google"):
        st.login() # hands off to Google's OAuth/OIDC flow

    # ...or the platform's own username/password system:
    AuthService().login(username=username, password=password)
    st.session_state["username"] = username
```

Two completely separate identity systems, both landing in the same gate: Google OAuth (`st.login()`, using Streamlit's built-in OIDC support) needs no Postgres row at all — it's just a signed token from Google. The platform's own system (`AuthService`) is your own `User` table with hashed passwords. Right now, *neither* path actually changes which account's data gets displayed — see Section 4.4 and the earlier discussion on this: today, login is a viewing gate, not a data boundary.

9.2 Auto-refresh without a full page reload

```
@st.fragment(run_every=REFRESH_SECONDS)
def live_view() -> None:
    state, trades = load_logs()
    ...
```

`st.fragment(run_every=3)` re-runs *just this function* every 3 seconds, rather than reloading the entire page — this is what makes numbers update live without the whole browser tab flickering/resetting.

10. Direct Answer: Can This System Handle Multiple Strategies From Different Users?

Split honestly into two halves:

The C++ engine: yes, already. `UserStateStore`'s per-user locking (Section 4) and gRPC's built-in worker-thread pool mean two different users' orders can already be checked and executed on separate threads, genuinely concurrently, safely. Nothing needs to change here for multi-user support.

The Python layer driving it: not yet. `strategy_runner.py` is a single sequential loop hardcoded to one account. Making it handle multiple researchers means (a) giving every login a real funded account, (b) having the runner track one independent strategy "memory" per user instead of one global one, and (c) not re-checking the same shared price separately for every single user when they're all trading the same symbol. None of that requires new locking or threading tricks in C++ — the concurrency-safe foundation is already sitting there waiting to be used from the Python side.

11. Glossary

- **Mutex / lock** — a mechanism ensuring only one thread touches a piece of shared data at a time, preventing corrupted results from simultaneous access.
- **gRPC** — a framework for calling functions across different programs/languages over a network, using a `.proto` file as the shared contract.
- **WAL (Write-Ahead Log)** — writing down what happened to a simple append-only file immediately, before/as it's processed further, so nothing is lost if something crashes.
- **Kafka** — a durable message queue. Producers drop messages in; consumers read them whenever they're ready, independently of each other's timing.
- **OIDC / OAuth** — OpenID Connect, a standard protocol for "let this user prove who they are via Google/Microsoft/etc." rather than your own password system.
- **Bid / Ask / Spread** — the highest price buyers are offering / the lowest price sellers are asking / the gap between them.

- **Race condition** — a bug where the result depends on the unpredictable timing of two threads touching the same data, avoided here with per-user/per-symbol locks.